先进计算与数字工程研究中心

PATTERN RECOGNITION AND MACHINE LEARNING

Lecture 02 – From Lunch Orders to Data Analysis

Chizhi Chris ZHANG

zhangchizhi@ciomp.ac.cn

Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

Today: calculate, clean and explain the same lunch orders

Period	One dataset through the workflow
1 (45 min)	Run and trace Python. Calculate with arrays, shapes, selections and matrix products.
2 (45 min)	Read and clean labeled records. Keep units, identities, missing outcomes and join relationships explicit.
3 (45 min)	Interpret the same waits with summaries and plots. Check missingness, mixtures, association and the scope of conclusions.

Outputs

A runnable calculation, a cleaned table and an answer whose numerical claims can be traced to specific orders.

The lunch question starts with a concrete observation table

Order ID	Stall	Queue	Wait / min
101	A	2	4
105	" A "	8	12
107	A	3	missing
108	B	6	10
108	B	6	10
109	A	4	−1

- One row should represent one order. The raw CSV has 13 rows and 6 columns.
- A short queue may still move slowly. Queue length is a candidate input, while realized waiting time is an outcome.
- Spaces, a copied row and a missing-value code can change computed summaries.
- This small invented table supports classroom calculation. It is not a survey of campus service.

A data dictionary fixes the unit of each record

Field	Recorded when	Meaning
order_id	Order creation	Identity of one completed order record.
queue	Ordering	People ahead, measured by one counting rule.
wait_min	Pickup	Minutes from ordering to pickup; -1 means unrecorded.
price	Price known	Quoted amount, with an agreed currency.

Why this precedes calculation

A parser can read values successfully without knowing their units or missing-value conventions. The dictionary distinguishes identifiers, inputs and outcomes before cleaning begins.

Python tools have complementary jobs in one experiment

Tool	Role	Lunch example
Python	Language and control flow	Write a function for average wait
NumPy	Numerical arrays	Convert all known waits to seconds
pandas	Labeled tables	Compare waits by stall
Matplotlib	Plotting	Show a histogram of recorded waits

Execution environment

A Python interpreter runs the code. Packages add tools to that interpreter. An editor or notebook provides a place to write and inspect it.

A common error

Installing pandas for one interpreter and running another can produce an import failure even when installation succeeded.

One environment makes installation and execution consistent

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install numpy pandas matplotlib
.\.venv\Scripts\python.exe -c "import numpy,pandas; print('Ready')"
```

Run these in the Windows terminal

Work in your course directory. The first command creates a local environment. The next commands use the Python inside it.

- Install packages and run scripts with the same interpreter path.
- In Python, `import sys; print(sys.executable)` shows the interpreter actually running.
- Select that environment in your editor or notebook as well.

A successful import and a correct data path are separate checks

```
import sys
from pathlib import Path
import pandas as pd
print(sys.executable)
print(Path.cwd())
data_file = Path("orders_raw.csv").resolve()
print(data_file, data_file.exists())
```

- The interpreter path identifies which environment runs the program.
- The current directory controls a relative file path. It need not be the script directory.
- If pandas imports but the file is absent, inspect the resolved data location rather than reinstalling packages.
- Keep the supplied script and CSV together, or use one explicit project data directory.

Variables and functions express a calculation with reusable meaning

```
waits = [4, 6, 3, 9]          # minutes
def average_wait(values):
    return sum(values) / len(values)
mean_min = average_wait(waits)
print(mean_min, mean_min * 60)  # 5.5, 330.0
waits.append(13)
print(average_wait(waits))      # 7.0
```

Check by hand

First total $4 + 6 + 3 + 9 = 22$ and divide by 4. After adding 13, total 35 and divide by 5.

- A variable names a value. Assignment updates the value used later.
- A function receives inputs and returns a result for further calculation.
- This version requires a nonempty collection of known waits in the same unit.

A loop processes observations in a visible sequence

```
total = 0
count = 0
for wait in [4, 6, 3, 9]:
    total += wait
    count += 1
print(total, count, total / count)  # 22, 4, 5.5
```

After reading	Total	Count	Meaning
4	4	1	One observed wait.
6	10	2	Two waits contribute.
3	13	3	Order remains meaningful.
9	22	4	The final mean is 22/4.

Conditions define exactly which waits enter a calculation

```
selected = []
for wait in [4, 6, 3, 9]:
    if wait > 5:
        selected.append(wait)
print(selected)                  # [6, 9]
print(sum(selected)/len(selected))  # 7.5
```

- The condition is checked for every observation. Only True cases enter the selected collection.
- Strictly greater than 5 differs from at least 5.
- The selected mean describes two slower orders, while the original mean describes all four.
- The program must handle an empty selected collection before dividing by its length.

A function contract states what an average expects

```python
def average_or_none(values):
    if len(values) == 0:
        return None
    return sum(values) / len(values)
print(average_or_none([6, 9]))  # 7.5
print(average_or_none([]))      # None
```

- This function expects a list of known numerical waits measured in the same unit.
- None signals that no mean is available. It is not a measured zero.
- An input contract also needs a policy for missing entries and invalid values.
- Returning a result lets later code decide whether to report it, plot it or investigate unavailable data.

A script runs in order, while a notebook keeps state between cells

```
# Cell 1
total = 22
# Cell 2: run this twice without rerunning Cell 1
total = total + 4
print(total)  # first: 26; second: 30
```

Script	Start at the top and execute sequentially. Save as `lunch.py`.
Notebook	Cells can run in a different order. Variables retain earlier values.

A reproducible result

Restart the notebook kernel and run all cells in order. If the answer changes, examine which previous state the calculation depended on.

File context

A script can locate data relative to its own file. A notebook commonly reads relative to its current working directory.

A notebook exercise distinguishes assignment from accumulation

```
# Cell A
waits = [4, 6, 3, 9]
# Cell B: repeat this cell
mean_wait = sum(waits) / len(waits)
# Cell C: repeat this cell
waits.append(13)
```

Repeated cell	Effect	How to check
B	Recomputes 5.5 from the same list	The list length stays 4.
C	Adds another 13 each time	Print waits and its length.

An error message identifies what the program could not do

Error	Likely immediate issue	Concrete check
`ModuleNotFoundError`	Spelling or interpreter	`numpy`, then interpreter path
`FileNotFoundError`	File location	Actual working directory and full path
`NameError`	Undefined variable	Assignment and notebook execution order
`ValueError`	Incompatible values or shapes	Input values, array shapes and operation

Reading a traceback

Read the final line for the error type. Then locate the line of your own code that caused it. Compare what that operation requires with the object supplied.

Deliberate typo

`import numppy` fails because the package is called `numpy`. Reinstalling the same package will not repair that spelling.

Tracing intermediate values locates an incorrect unit conversion

```
wait_min = [4, 6, 3, 9]
mean_min = sum(wait_min)/len(wait_min)
mean_seconds = mean_min * 60
print(len(wait_min), mean_min, mean_seconds)
# 4, 5.5, 330.0
# Incorrect: mean_min * 100 is not minutes to seconds.
```

- Inspect input count, intermediate mean and final unit separately.
- A plausible-looking number can result from the wrong conversion factor.
- Changing only output formatting does not repair the calculation.
- For unfamiliar data, check a few records manually before trusting a bulk operation.

Array arithmetic depends on the object type

```
import numpy as np
print([4, 6] * 2)                  # [4, 6, 4, 6]
wait = np.array([4, 6, 3, 9], dtype=float)
print(wait * 60)                   # [240. 360. 180. 540.]
print(wait.mean())                 # 5.5
```

A list and an array do different work

A list repeats its contents under multiplication by an integer. A numerical array multiplies each stored value.

Units make the intention clear

Converting minutes to seconds changes each waiting value. Repeating the list instead creates more entries without performing the conversion.

Vectorization

One array expression applies an operation to many values while preserving their order. Always check the object's type and output shape.

Array constructors make shape and initialization explicit

```
a = np.arange(6)              # [0, 1, 2, 3, 4, 5]
B = a.reshape(2, 3)
Z = np.zeros((2, 3), dtype=float)
O = np.ones((2, 3))
print(B.shape, Z.dtype)       # (2, 3), float64
```

- A sequence constructor specifies values. Reshape organizes the same six values into two rows.
- Zeros and ones initialize every element with a known value.
- Shape records axes, while dtype specifies how each value is stored.
- A numerical example should state whether rows are observations, pixels or simply positions for learning an operation.

Array shape and indexing keep each observation identifiable

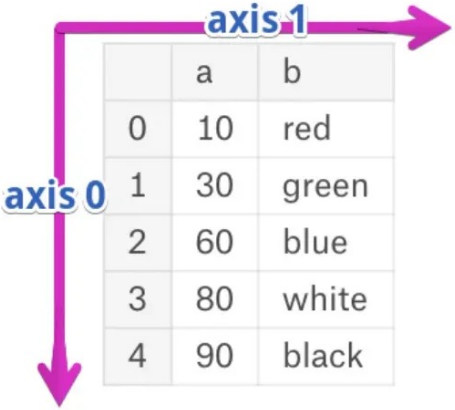
```
X = np.array([[2, 4], [4, 6], [1, 3], [5, 9]], dtype=float)
print(X.shape, X.ndim)   # (4, 2), 2
print(X[0, 1])           # 4.0
print(X[:2, :])          # [[2., 4.], [4., 6.]]
print(X[:, 1])           # [4., 6., 3., 9.], shape (4,)
print(X[:, 1:2].shape)   # (4, 1)
```

Meaning of the two columns

Queue length and wait in minutes. Rows represent four orders. Python indexes from zero and excludes a slice's stop position.

- `ndim` counts axes. A 4×10 array still has two axes.
- A single-column selection can remove an axis. A slice can preserve it.

Array shape and indexing keep each observation identifiable



	a	b
0	10	red
1	30	green
2	60	blue
3	80	white
4	90	black

知乎 @白马银枪

Sample count, feature count and array rank differ

Object	Shape	Rank	Meaning
One feature for 4 orders	$(4,)$	1	Four values along one axis.
Four orders, two features	$(4, 2)$	2	Samples and features have separate axes.
Two grayscale images	$(2, 28, 28)$	3	Sample, image row and image column.
Two RGB images	$(2, 28, 28, 3)$	4	A channel axis is included.

Two uses of dimension

A 784-feature dataset can be a rank-2 array $(n, 784)$. Many features do not imply many array axes. State which meaning of dimension you use.

Paired indexing differs from selecting complete rows

```
print(X[[0, 3]])        # rows 0 and 3, shape (2, 2)
print(X[[0, 3], [1, 0]])  # [4., 5.]
print(X[np.ix_([0, 3], [0, 1])])  # a rectangular selection
```

- Two index arrays select paired coordinates: $(0, 1)$ and $(3, 0)$ here.
- A single row-index array selects complete observations.
- np.ix_ constructs a row-by-column rectangular selection.
- The values 4 and 5 in the paired result come from different columns with different units, so their average would not describe a useful waiting quantity.

Boolean selection and sorting must preserve complete rows

```
wait = X[:, 1]
mask = wait > 5
print(mask)               # [False, True, False, True]
print(X[mask])            # [[4., 6.], [5., 9.]]
positions = np.argsort(wait)
print(X[positions])       # complete rows ordered by wait
print(mask.sum(), mask.mean())  # 2, 0.5
```

One mask describes one set of observations

Select the entire row using the condition on its waiting time. Sorting positions similarly reorder entire rows.

Independent column sorting invents observations

The column means may stay unchanged even after queue and wait pairs have been broken. A relationship calculation on the altered table then describes nonexistent orders.

A compound array mask keeps the selected rows together

```
queue, wait = X[:, 0], X[:, 1]
keep = (queue >= 2) & (wait <= 6)
print(keep)  # [True, True, False, False]
print(X[keep])  # [[2., 4.], [4., 6.]]
```

- The criterion is at least two people ahead and at most six minutes of waiting.
- Each comparison produces one Boolean per order. Parentheses keep the comparisons distinct.
- Use elementwise & and | for arrays, rather than Python and or or for entire arrays.
- Select complete rows so later relationships still refer to the original orders.

A reduction removes an axis, but units still determine meaning

```
print(X.mean(axis=0))          # [3.0, 5.5]
two_visits = np.array([[4, 6], [3, 9]])
print(two_visits.mean(axis=1)) # [5.0, 6.0]
```

Operation	What gets combined	What remains
`X.mean(axis=0)`	Orders along the row axis	One mean for each feature
`mean(axis=1)`	Two visits along the column axis	One mean for each person

Shape-valid need not be meaningful

Averaging queue length with waiting minutes returns a number but mixes incompatible units. The two visits have the same unit, so their row average is interpretable.

Broadcasting reuses values only along compatible dimensions

```
center = X.mean(axis=0)  # [3., 5.5], shape (2,)
print(X - center)       # [[-1., -1.5], [1., .5],
                        #  [-2., -2.5], [2., 3.5]]
per_row = np.array([1, 2, 3, 4]).reshape(4, 1)
print(per_row.shape)  # (4, 1), shape demonstration
```

Shape rule

Compare dimensions from the right. Corresponding sizes must be equal or one must be 1. A missing leading dimension acts like 1.

Shapes	Compatible?	Result
$(4, 2)$ and $(2,)$	Yes	$(4, 2)$
$(4, 2)$ and $(4, 1)$	Yes	$(4, 2)$
$(4, 2)$ and $(4,)$	No	Last dimensions 2 and 4 disagree

A broadcasting exercise checks the intended orientation

Shapes	Valid?	What the smaller object supplies
$(4, 2)$ and $(2,)$	Yes	One value per column.
$(4, 2)$ and $(4, 1)$	Yes	One value per row, expanded across columns.
$(4, 2)$ and $(4,)$	No	The last dimensions 2 and 4 disagree.
$(4, 2)$ and $(1, 2)$	Yes	An explicit single row repeats.

Before execution

You want to center each feature. Which shape should the two feature means have? Explain why four row-specific values require an explicit singleton column axis instead.

Geometry and meaning

A shape-compatible subtraction still needs matching units for each feature.

Elementwise products and matrix products calculate different answers

```
A = np.array([[1, 2], [3, 4]])
v = np.array([10, 20])
print(A * v)  # [[10, 40], [30, 80]]
print(A @ v)  # [50, 110]
```

$$(A\mathbf{v})_1 = 1(10) + 2(20) = 50, \qquad (A\mathbf{v})_2 = 3(10) + 4(20) = 110.$$

- `*` multiplies corresponding values with broadcasting. It keeps two independent products in each row.
- `@` sums feature contributions within each row, giving one result per observation.
- A linear model commonly uses `X @ w + b`. Its output shape follows the number of observations.

Check one row

Both operations run successfully. A hand calculation and a shape check distinguish them.

A matrix product predicts one score for each sample

```
features = np.array([[2., 1.], [4., 3.], [1., 2.]])
w = np.array([1., -0.5])
b = 2.
scores = features @ w + b
print(scores)  # [3.5, 4.5, 2.0]
```

$$s_1 = 2(1) + 1(-0.5) + 2 = 3.5.$$

- Rows are three observations and columns are two comparable input features in a separate calculation.
- A weight belongs to a feature, not to a sample.
- The output has three values. A later decision rule can turn scores into predicted classes.
- Check output shape and manually verify one row before increasing the dataset size.

Reshaping changes organization, while a view may share stored values

```
pixels = np.zeros((28, 28))
flat = pixels.reshape(-1)          # shape (784,)
restored = flat.reshape(28, 28)    # same number of values
a = np.array([4., 6., 3., 9.])
b = a[:2]                          # a basic slice shares data
b[0] = 99
print(a)                           # [99., 6., 3., 9.]
safe = a[:2].copy()                # independent array
```

Shape is part of the representation

Reshaping preserves element count and uses a specified order. It does not learn image features or create additional information.

Unexpected mutation

Changing a shared slice can change the original data. Make an explicit copy when a separate experiment should leave the source values intact.

Data types and missing values affect a numerical result

```
wait = np.array([4., 6., np.nan, 9.])
print(np.isnan(wait))  # [False, False, True, False]
print(wait.mean())     # nan
print(np.nanmean(wait))  # 19/3 = 6.333...
```

- `np.nan` represents a missing floating-point value. It is not a zero-minute wait.
- The ordinary mean signals missing input here. `nanmean` divides the observed sum by the observed count.
- Choose a type that can represent intended values. Small unsigned image integers can overflow under arithmetic.
- A result computed after skipping missing values describes only the observed subset unless further assumptions justify a broader interpretation.

Next stage

pandas lets us retain the order identifier and stall while recording missing waits explicitly.

A DataFrame preserves column names and the observation unit

```
from pathlib import Path
import pandas as pd
data_dir = Path(__file__).resolve().parent  # script location
orders = pd.read_csv(data_dir / "orders_raw.csv")
print(orders.shape)      # (13, 6)
print(orders.head())
orders.info()
```

Read the schema before cleaning

One row is one recorded order. The columns are ID, stall, order time, queue length, waiting minutes and price.

- `head()` is a preview. `info()` summarizes the whole table's types and nonmissing counts.
- In a notebook, use the known data directory or inspect `Path.cwd()`. The name `__file__` is usually unavailable.

CSV parsing can succeed with the wrong table structure

Symptom	Possible cause	Concrete check
13 rows, 1 column	The delimiter does not match the file	Inspect one original text line and specify sep.
First order appears as column names	Header convention differs	State whether a header exists.
Unreadable category text	Encoding mismatch	Use the file's stated encoding.
Unexpected extra column	An exported index was included	Inspect column names before dropping anything.

Successful reading is a structural check

Confirm rows, columns, names and a few known values. A DataFrame object alone does not prove the intended schema was recovered.

Column selection, row labels and positions are different selectors

```
print(orders["wait_min"].head())          # one Series
print(orders[["queue", "wait_min"]].head()) # DataFrame
print(orders.loc[0:2, ["stall", "wait_min"]]) # labels 0,1,2
print(orders.iloc[:2, 1:3])              # positions 0,1
```

Selection	Meaning	Important detail
Column names	Choose fields	A one-column list preserves a table
`loc`	Row and column labels	Label slicing includes the stop label
`iloc`	Integer positions	Positional slicing excludes the stop

An index label is not always a row number

After filtering or assigning a new index, labels can differ from positions. Read the selector according to its definition.

A duplicate index label can return several observations

```
by_id = orders.set_index("order_id")
print(by_id.loc[108])  # two rows before duplicate removal
print(by_id.loc[[101]])  # list selection keeps a DataFrame
print(by_id.index.is_unique)  # False in the raw table
```

- An index organizes labels. Assigning a column as the index does not establish uniqueness.
- A scalar label can match multiple rows when the index repeats.
- A one-item label list preserves a table-shaped selection.
- Determine whether repeated identities are copied records or legitimate events before choosing a correction.

Pandas arithmetic aligns labels rather than displayed positions

```
a = pd.Series([4, 6], index=[101, 102])
b = pd.Series([20, 10], index=[102, 101])
print(a + b)   # ID 101: 14; ID 102: 26
orders["order_id"].duplicated().sum()  # 1 repeated ID
```

Alignment preserves identities

For ID 101, pandas adds 4 to 10 even though those entries occupy different displayed positions. The index determines the match.

- Unmatched labels can produce missing results after alignment.
- Order IDs identify observations. They should not become a numerical predictor merely because they are integers.
- Duplicate IDs need interpretation. A repeated ID might be a copied record or several legitimate events in a different data design.

An alignment exercise matches values by identity

```
price = pd.Series([12, 15, 10], index=[101, 102, 103])
discount = pd.Series([3, 1], index=[102, 101])
paid = price - discount
print(paid)  # 101:11, 102:12, 103:NaN
```

- ID 101 pairs 12 with 1; ID 102 pairs 15 with 3 despite opposite display order.
- ID 103 lacks a corresponding discount value, so the aligned result is missing.
- Filling that missing result with zero would mean zero payment, not zero discount.
- If the policy defines missing discounts as none, fill the discount input before subtraction and explain the rule.

Category normalization and numeric parsing make hidden differences visible

```
orders["stall"] = orders["stall"].str.strip()
raw_wait = orders["wait_min"]
parsed = pd.to_numeric(raw_wait, errors="coerce")
parse_failed = raw_wait.notna() & parsed.isna()
print(orders["stall"].value_counts())
print(orders.loc[parse_failed, ["order_id", "wait_min"]])
orders["wait_min"] = parsed
```

Two different issues

Whitespace can split one category into several labels. Unparseable text can prevent numerical calculation.

- `strip()` maps `" A "` to `"A"` without inventing a new stall.
- `errors="coerce"` marks failed numeric conversion as missing. Inspect which values failed instead of silently treating them as valid observations.
- Keep the raw file so the transformed values remain traceable.

Parsing numerical text requires a unit rule

```
text = pd.Series(["4", " 6 ", "12 min", "unknown"])
normalized = text.str.strip().str.replace(r"\s*min$", "", regex=True)
values = pd.to_numeric(normalized, errors="coerce")
print(values.to_list())  # [4.0, 6.0, 12.0, nan]
```

- This example assumes every valid observation is measured in minutes and permits a trailing min label.
- Removing a documented suffix recovers a measurement. It does not recover an unknown value.
- Retain the original text and inspect conversion failures.
- A blanket removal of all nonnumeric characters can silently destroy signs, decimal points or mixed-unit information.

Mixed units need conversion before aggregation

Original text	Numerical value	Unit	Converted minutes
240 s	240	Seconds	4
6 min	6	Minutes	6
unknown	Missing	Unknown	Missing

$$\text{minutes} = \begin{cases} \text{value}/60, & \text{unit is seconds}, \\ \text{value}, & \text{unit is minutes}. \end{cases}$$

- Parse the number and unit separately, then apply an explicit conversion map.
- The mean of 4 and 6 is 5 minutes. The mean of raw numbers 240 and 6 is 123, which mixes units.
- Unknown units require investigation rather than an arbitrary conversion.

Parsing timestamps makes time operations explicit

```
orders["ordered_at"] = pd.to_datetime(
    orders["ordered_at"], errors="coerce")
print(orders["ordered_at"].isna().sum())  # inspect failures
orders["hour"] = orders["ordered_at"].dt.hour
print(orders[["order_id", "ordered_at", "hour"]].head())
```

- A timestamp combines a date and time. Its meaning is richer than a text string or a standalone hour.
- Parsing enables chronological sorting and time differences.
- A failed parse becomes `NaT`, which should be investigated.
- Extracting the hour answers a coarser question and discards within-hour order.

Our records

11:40 and 11:50 have the same extracted hour. They remain distinct order timestamps, so retain the original parsed column as well.

A time window is a representation with an explicit width

```
times = pd.to_datetime(["2026-09-01 11:40", "2026-09-01 12:05"])
slots = times.floor("30min")
print(slots)  # 11:30 and 12:00 on the same date
```

- Flooring groups timestamps into half-hour windows with a stated boundary convention.
- 11:40 falls in [11:30,12:00); 12:05 falls in [12:00,12:30).
- A window summarizes timing and discards within-window position.
- Choose a width suited to the question and data volume. This separate example does not change the original precise timestamp column.

Elapsed time requires the complete timestamps

```
ordered = pd.Timestamp("2026-09-01 23:58")
picked_up = pd.Timestamp("2026-09-02 00:06")
wait_min = (picked_up - ordered).total_seconds() / 60
print(wait_min)  # 8.0
```

Crossing midnight

Subtracting 58 from 6 gives -52, which is not the elapsed time. The dates show that pickup occurred the next day.

- Subtract timestamps to obtain a duration, then convert its unit.
- Check that pickup follows ordering and that clocks use the intended time convention.
- Retain timestamps for later checks even if the analysis uses a derived wait column.

Same calculation on a table

Subtract the pickup and order columns, then use `.dt.total_seconds()/60`.

A negative duration is evidence to investigate

Order time	Recorded pickup time	Duration	Interpretation
Sep 1, 23:58	Sep 2, 00:06	8 min	A valid cross-midnight pair.
Sep 1, 23:58	Sep 1, 00:06	-1432 min	Pickup precedes order; check the date.

- A negative time difference can result from an incorrect date, swapped fields or clock conventions.
- Taking its absolute value does not determine the true duration.
- Add a day only when the source convention and event evidence justify that correction.
- Keep original timestamps and flag the affected records for a documented decision.

Duplicate removal depends on what a row represents

```
copied = orders.duplicated(keep=False)
print(orders.loc[copied])        # both copies of order 108
orders = orders.drop_duplicates().copy()
print(len(orders))               # 12 distinct orders
print(orders["order_id"].nunique())  # 12
```

Case-specific interpretation

Here order 108 was copied as an identical full row. Removing one copy restores one row per order.

- Similar waits or equal prices do not make two different orders duplicates.
- If repeated IDs describe status updates, the proper action is to select or combine events according to that design.
- Inspect duplicate candidates before choosing which columns define duplication.

Statistical effect

Counting the same 10-minute order twice changes frequencies, means and scatter-plot multiplicities even when all values look plausible.

An observation key can combine date and order ID

Date	Order ID	Same event?
Sep 1	101	First day's order.
Sep 2	101	A different order if numbering restarts daily.
Sep 2	101	A possible duplicate of the second row.

Composite key

Under the daily-numbering convention, use (date, order ID) to identify an order. Checking only order ID would wrongly combine different days.

- A key definition belongs to the observation unit.
- When equal keys have conflicting outcomes, inspect revisions and provenance rather than blindly keep the first row.

A missing wait is an unknown outcome rather than an immediate pickup

```
orders.loc[orders["wait_min"] == -1, "wait_min"] = np.nan
print(orders["wait_min"].isna().sum())  # 2
print(orders["wait_min"].count())       # 10 observed waits
print(orders["wait_min"].mean())        # 11.2, skips missing
```

Meaning of the code

The teaching dataset uses -1 to indicate an unrecorded wait. Together with the blank cell, it produces two explicitly missing values.

- A zero-minute wait is a real possible observation. It should not substitute for an unknown value.
- Skipping missing values computes an observed-case mean.
- Filling values requires an additional rationale and changes the analysis.

Denominator

The recorded wait sum is 112 minutes. Dividing by 10 describes the recorded waits. Dividing by all 12 orders requires information about the other two outcomes.

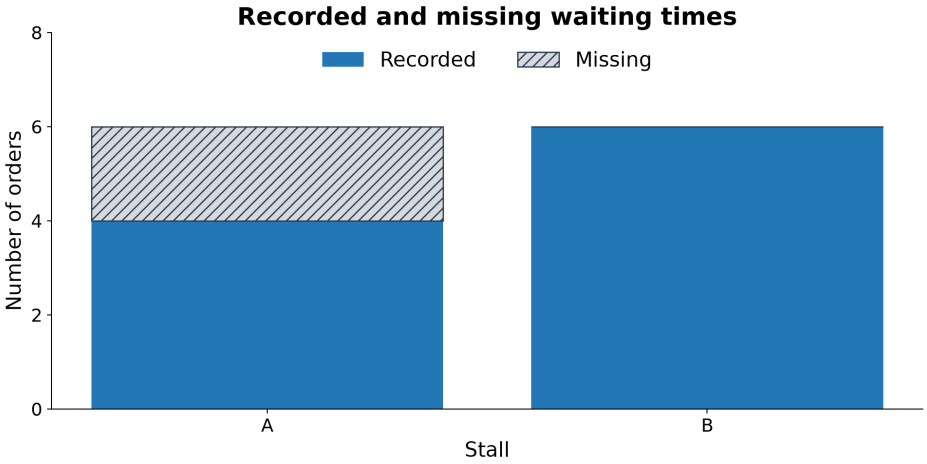
The pattern of missingness matters for an observed mean

Why a wait may be absent	Potential effect	What evidence is needed
Long-wait customers leave before recording	Observed waits may be too short	Check completion and recording process.
Quick orders are overlooked	Observed waits may be too long	Check how records were selected.
Random equipment loss	Less precision without a directional claim	Investigate whether loss depends on conditions.

Missingness is not a value

The same missing count can arise through different processes. A statistic based on observed values needs its population and assumptions stated.

The pattern of missingness matters for an observed mean



Mean imputation preserves a mean but reduces observed variation

```
s = pd.Series([4., 6., np.nan, 9.])  # separate sensitivity example
filled = s.fillna(s.mean())
print(s.mean(), filled.mean())  # both 6.333...
print(s.var(ddof=0), filled.var(ddof=0))  # 4.222..., 3.166...
```

- The filled value lies exactly at the observed mean, so the mean stays fixed.
- The squared deviations are unchanged but their divisor increases from 3 to 4.
- This invented outcome must not replace an unknown measurement in the official observed-case report.
- For predictor inputs, fit an imputation rule on training data and consider how its uncertainty affects later analysis.

One full table and one observed subset support different questions

```
orders.loc[orders["wait_min"] < 0, "wait_min"] = np.nan
valid = orders.loc[orders["wait_min"].notna()].copy()
print(len(orders), len(valid))   # 12, 10
print(orders.groupby("stall")["wait_min"].size())
print(valid.groupby("stall")["wait_min"].size())
```

Two legitimate populations

The full cleaned table retains all orders and missingness information. The observed subset supports calculations that require known waits.

- Explicit `loc[row condition, column]` assignment changes the intended original table.
- A copy of a selected subset allows separate calculations without unexpected shared changes.
- State whether each statistic uses all orders or only observed waits.

Dropping early hides evidence

Removing missing rows from the only table loses the ability to describe which stall lacks outcomes.

An explicit assignment avoids an uncertain temporary target

```
example = pd.DataFrame({"x": [1, 2, 3]})
# Ambiguous intent: example[example["x"] > 1]["x"] = 0
example.loc[example["x"] > 1, "x"] = 0
print(example["x"].to_list())  # [1, 0, 0]
```

- The intended modification targets the original table and a particular column.
- A chained expression may refer to a temporary selection; behavior and diagnostics vary by pandas version.
- Use .loc for a deliberate write to the original object.
- Use an explicit copy for an independent experiment, and inspect the result after a mutation.

Combined conditions answer a precise row-level question

```
mask = (valid["stall"] == "A") & (valid["wait_min"] > 5)
slow_a = valid.loc[mask, ["order_id", "queue", "wait_min"]]
print(slow_a)
# 102: queue 4, wait 6; 105: queue 8, wait 12
print(slow_a["wait_min"].mean())  # 9.0
```

Read the condition aloud

Among recorded waits, select orders at stall A that took strictly more than 5 minutes.

- Parentheses delimit each column-wise comparison. `&` combines their Boolean results.
- `and` is not the elementwise operator for pandas Series.
- $>$ and $\geq$ answer different questions when a wait equals 5.
- Compute the mean only after identifying the selected rows and their denominator.

Group size and observed count report different evidence

```
summary = orders.groupby("stall").agg(
    all_orders=("order_id", "size"),
    observed=("wait_min", "count"),
    mean_wait=("wait_min", "mean"),
    median_wait=("wait_min", "median"))
print(summary)
```

Stall	All	Observed	Mean / min	Median / min
A	6	4	6.75	5.50
B	6	6	14.17	12.50

Denominators have names

`size` counts rows, including missing waits. `count` counts nonmissing values in the selected column. Mean and median use those observed values.

A preliminary comparison

The observed mean is lower at A. Its two missing waits leave uncertainty about the full-order comparison.

A weighted-mean exercise preserves group sample counts

Group	Observed count	Mean / min	Total wait / min
One	2	4	8
Two	3	6	18
Three	5	12	60

$$\overline{w} = \frac{2(4) + 3(6) + 5(12)}{2 + 3 + 5} = 8.6 \text{ min.}$$

- The ordinary average of the three group means is $22/3 \approx 7.33$, answering a group-weighted question instead.
- The observation-weighted mean reconstructs the total sum and total count.
- These groups form a separate teaching example rather than additional lunch orders.

The two stall means can be checked directly from their orders

Stall	Recorded waits / min	Mean
A	$4, 6, 12, 5$	$27/4 = 6.75$
B	$3, 9, 15, 10, 18, 30$	$85/6 \approx 14.17$

$$\text{Overall observed mean} = \frac{27 + 85}{4 + 6} = 11.2 \text{ min.}$$

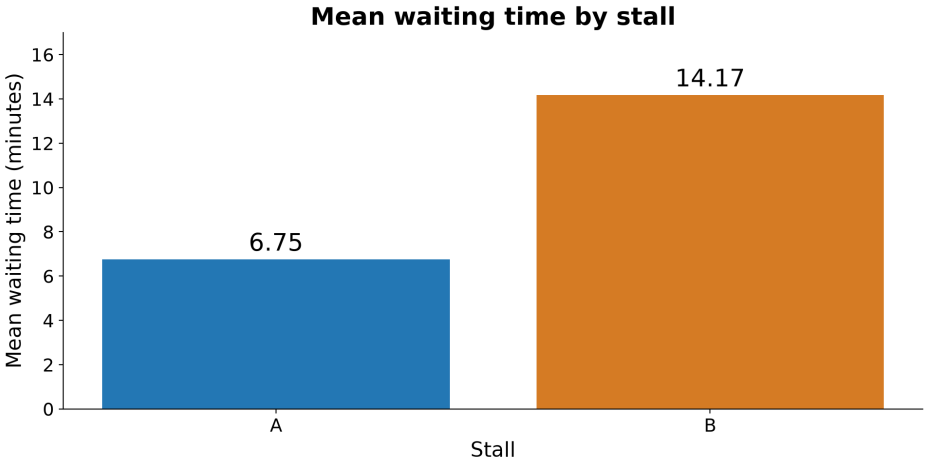
Group averages require weights

The overall mean weights the stall means by their observed counts: $(4 \cdot 6.75 + 6 \cdot 14.1667)/10$. The unweighted average of the two means answers a different question.

Interpretation

A contributes four observed waits and B contributes six. Preserve that fact when moving from grouped summaries to a combined result.

The two stall means can be checked directly from their orders



Cross-tables distinguish counts from conditional proportions

```
late = valid["wait_min"] > 10
counts = pd.crosstab(valid["stall"], late)
rates = pd.crosstab(valid["stall"], late, normalize="index")
print(counts)
print(rates)
```

	$\leq$ 10 min	$>$ 10 min	Observed	Share $>$ 10
A	3	1	4	25%
B	3	3	6	50%

Conditional question

Row normalization divides by the observed count at each stall. It estimates the share exceeding 10 minutes among that stall's recorded waits.

Different denominator, different meaning

Column normalization instead asks how each threshold group divides across stalls. Two missing A waits do not enter either observed-wait table.

Row and column normalization answer different questions

Recorded orders	Not late	Late	Row total
A	3	1	4
B	3	3	6
Column total	6	4	10

$$P(\text{late} \mid B) = 3/6 = 0.5, \qquad P(B \mid \text{late}) = 3/4 = 0.75.$$

- The first question selects B orders, then asks which are late.
- The second selects late orders, then asks which come from B.
- A row-normalized and column-normalized table should each be labeled with its conditioning direction.
- This is the same denominator distinction that appears in class probabilities and confusion-matrix metrics.

A merge joins attributes by a key and can multiply observations

```
stalls = pd.read_csv(data_dir / "stalls.csv")
enriched = orders.merge(stalls, on="stall", how="left",
                        validate="many_to_one", indicator=True)
print(len(enriched))    # 12 when every right-side key is unique
print(enriched["_merge"].value_counts())
```

- Each order gets the food attribute of its stall. Many orders may match one stall record.
- Two right-side records with key A would make each A order appear twice in the joined result.
- `validate="many_to_one"` rejects that unintended duplication.
- A left join keeps orders with unmatched keys and marks their added attributes as missing.

Row meaning is the invariant

After the join, one row should still represent one order. Check counts and unmatched keys before calculating another average.

A duplicated right-side key visibly multiplies rows

```
left = pd.DataFrame({"order_id": [201, 202], "stall": ["A", "A"]})
right = pd.DataFrame({"stall": ["A", "A"], "food": ["Noodles", "Other"]})
joined = left.merge(right, on="stall")
print(len(joined))  # 4: each order matches both right rows
```

- Two left observations times two matching right records produce four joined rows.
- The join is valid relationally but violates the intended one-row-per-order enrichment.
- validate="many_to_one" would reject this right table.
- Correct the source relation or deliberately define a different observation unit before aggregating the result.

Concatenation adds observations, while export preserves the clean schema

```
# next_day has the same columns and row meaning
combined = pd.concat([orders, next_day], ignore_index=True)
orders.to_csv(data_dir / "orders_clean.csv", index=False)
summary.to_csv(data_dir / "stall_summary.csv")
```

Operation	Intended change
Concatenate rows	Add another set of orders with compatible column definitions
Merge by a key	Add attributes belonging to each existing order
Export cleaned table	Store values and column names without a stray index column

An identifier still needs a policy

Concatenating days can create repeated order IDs if numbering restarts each day. A date-plus-ID key may be necessary. The displayed index and the real order identifier have different roles.

Short exercise: explain each change from raw rows to valid waits

Stage	Expected result and reason
Raw file	13 rows, including one copied order
Remove exact copy	12 distinct orders
Normalize missing code	Two unknown waits: orders 107 and 109
Select observed waits	10 orders used for wait summaries
Filter A and wait > 5	Orders 102 and 105, mean $(6 + 12)/2 = 9$ min

Explain before running

For each operation, state which observations stay, what changes, and why the calculation's denominator follows from that choice.

Bridge to exploration

The next period uses the same ten observed waits. Every histogram bar and summary must trace back to these rows.

A mean and a median describe different aspects of the same waits

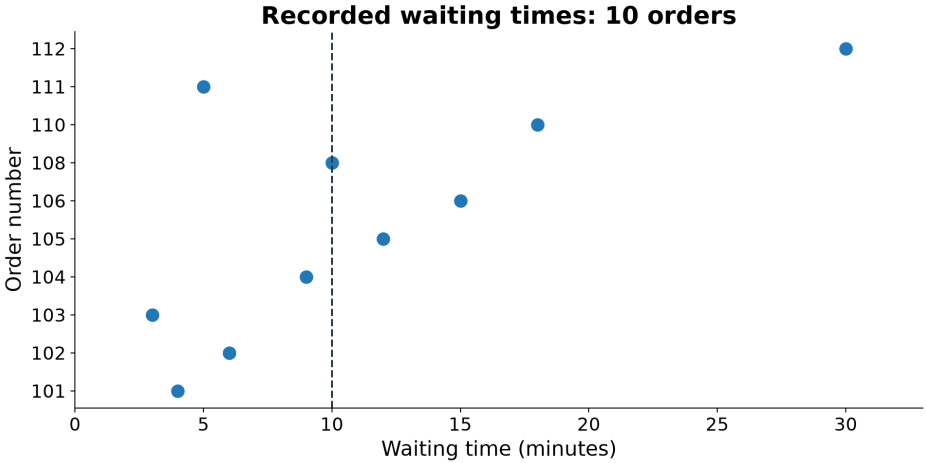
$3, 4, 5, 6, 9, 10, 12, 15, 18, 30$ minutes, sorted.

Summary	Calculation	Interpretation
Mean	$112/10 = 11.2$ min	Total waiting burden per recorded order
Median	$(9 + 10)/2 = 9.5$ min	Middle position among recorded waits
Range	$30 - 3 = 27$ min	Observed longest-minus-shortest wait
Share > 10	$4/10 = 40\%$	Recorded orders exceeding a target

A number answers a question

The 30-minute order affects the sum strongly. The median instead depends on the middle positions. Report summaries that reflect the aspect of waiting you need to understand.

A mean and a median describe different aspects of the same waits



A descriptive table reports count and scale conventions

```
description = valid["wait_min"].describe()
print(description[["count", "mean", "std", "min", "50%", "max"]])
```

Entry	What it refers to
count = 10	Nonmissing waiting outcomes, not all 12 orders.
mean = 11.2	The observed sum 112 divided by 10.
std	Spread using pandas' default divisor n-1.
50% = 9.5	Median under the stated quantile convention.

Read a statistic as a sentence

State the variable, sample subset, unit and calculation convention before comparing results from different tools.

A sensitivity calculation shows how one long wait changes the mean

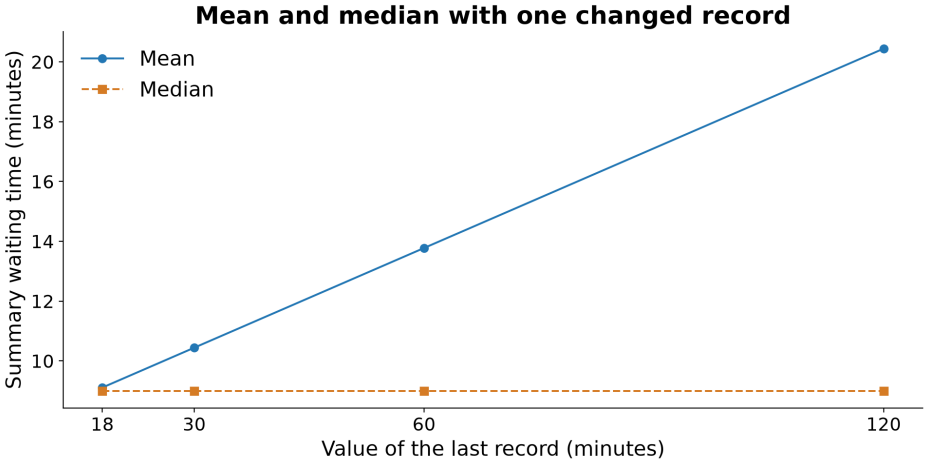
A separate nine-wait experiment. Eight fixed waits are $3, 4, 5, 6, 9, 10, 12, 15$ (sum 64). Only the ninth value $v \geq 15$ changes.

$$\text{mean}(v) = \frac{64 + v}{9}, \qquad \text{median}(v) = 9.$$

Ninth value v / min	Mean / min	Median / min
18	9.11	9
30	10.44	9
60	13.78	9
120	20.44	9

- Increasing v by 9 raises the mean by 1. The middle rank stays fixed, so the median remains 9.
- The mean describes total burden; the median describes the central position.
- A valid long wait still matters for service failures. Choose a summary according to the question.

A sensitivity calculation shows how one long wait changes the mean



A trimmed mean changes the population being summarized

Sorted waits $= 3, 4, 5, 6, 9, 10, 12, 15, 18, 30$.

Remove one value from each end: $(112 - 3 - 30)/8 = 9.875$ min.

- The full mean remains 11.2 and the median remains 9.5 for the original ten observations.
- A trimmed mean reduces the influence of both extremes by changing which observations contribute.
- The 30-minute order is still a valid service outcome and matters for tail experience.
- Name the trimming rule explicitly. Do not present the smaller result as the unchanged full-order average.

Histogram counts depend on explicit interval boundaries

```
import matplotlib.pyplot as plt
plt.hist(valid["wait_min"], bins=range(0, 36, 5))
plt.xlabel("Waiting time (minutes)")
plt.ylabel("Number of recorded orders")
plt.tight_layout()
```

Bin	$[0,5)$	$[5,10)$	$[10,15)$	$[15,20)$	$[20,25)$	$[25,30)$	$[30,35]$
Count	2	3	2	2	0	0	1

Read one bar

The interval $[5,10)$ contains 5, 6 and 9 minutes. A wait of 10 belongs to the next bin. Counts sum to the ten recorded orders.

Binning is a modeling choice

Wide bins hide detail. Very narrow bins fragment a small sample. Examine whether a claimed pattern survives reasonable changes to bin width.

Histogram counts depend on explicit interval boundaries

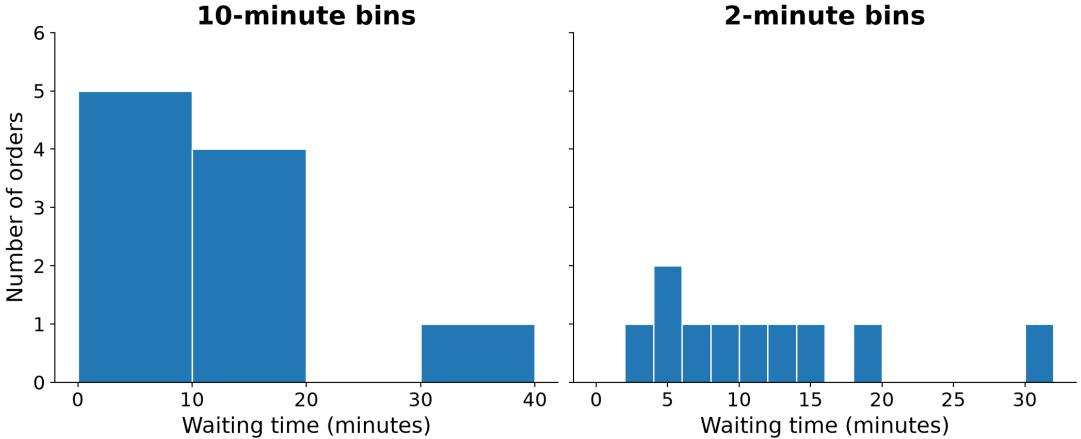


Bin-edge observations test a histogram specification

Wait / min	Five-minute bin	Reason
4.9	$[0, 5)$	Below the upper edge 5.
5.0	$[5, 10)$	The left edge is included.
10.0	$[10, 15)$	The previous upper edge is excluded.
35.0	Final bin if specified through 35	Matplotlib includes the rightmost edge of the last bin.

- Edge rules determine counts even when observations differ by very little.
- State the full bin sequence and the last-edge convention.
- For our ten recorded waits, check that counts still sum to ten.
- A plot clipped at a smaller upper range can silently omit a long wait.

Bin-edge observations test a histogram specification



Density uses bar area when histogram widths differ

$$\text{density in bin } j = \frac{\text{count}_j}{n\,\text{width}_j}.$$

Wait interval / min	Count	Width	Density / min	Area
$[0, 5)$	2	5	0.04	0.20
$[5, 10)$	3	5	0.06	0.30
$[10, 20)$	4	10	0.04	0.40
$[20, 35]$	1	15	0.0067	0.10

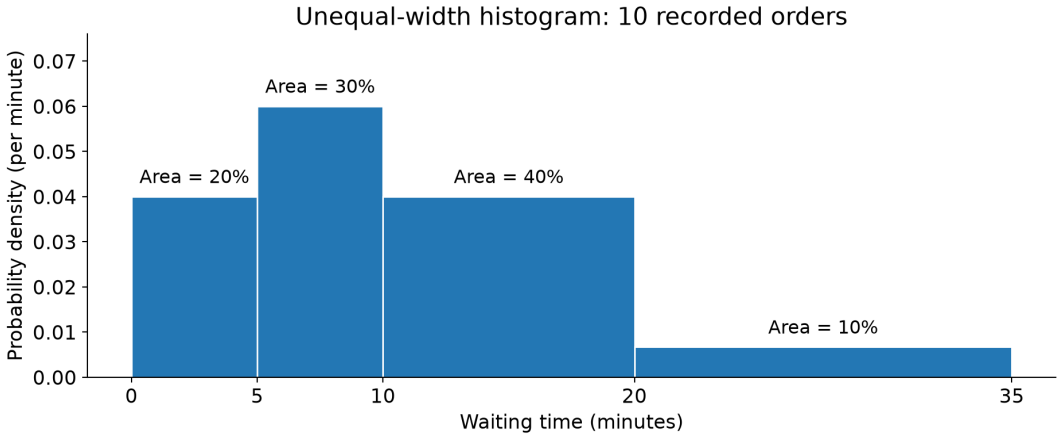
Area is the probability share

For the 10–20 minute bin, height 0.04 times width 10 gives 0.40, representing four of ten observations. Areas sum to one.

Height alone can mislead

The first and third bins have equal density heights but contain different observation shares because their widths differ. A density has an inverse-unit scale.

Density uses bar area when histogram widths differ



A density exercise reconstructs counts from area

Bin	Density per min	Width / min	Share	Count among 10
$[0, 5)$	0.04	5	0.20	2
$[10, 20)$	0.04	10	0.40	4
$[20, 35]$	0.00667	15	0.10	1

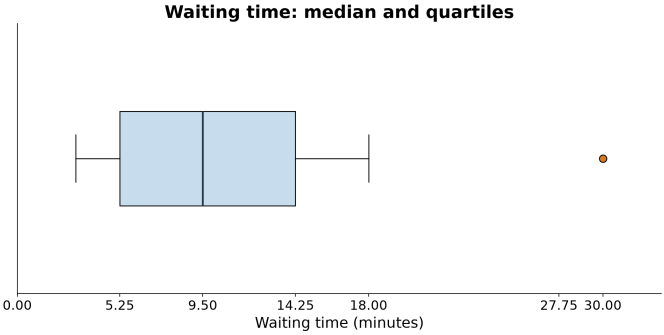
Three compatible views

Density times width gives a probability share. Multiplying the share by the sample count recovers the frequency. A taller bar alone need not contain more observations when bin widths differ.

Unit check

Inverse minutes times minutes gives a unitless share, then multiplying by 10 gives an observation count.

Quartiles and the interquartile range locate the middle half



Quantity	Minutes
Q_1	5.25
Median Q_2	9.50
Q_3	14.25
$IQR = Q_3 - Q_1$	9.00
Upper fence $Q_3 + 1.5IQR$	27.75

Box and line

The box spans the middle half. The internal line marks the median.

- Whiskers reach observed values 3 and 18. The 30-minute wait exceeds the upper fence and appears separately.
- These quartiles use pandas' linear interpolation. A flagged point invites investigation. It does not justify deleting a valid long wait.

Linear interpolation explains the fractional quartiles

$$h = (n-1)q = 9q \quad \text{with zero-based sorted positions.}$$

Quantile	Position h	Neighbor values	Interpolated result
$q = .25$	2.25	Position 2: 5; position 3: 6	$5 + .25(6-5) = 5.25$
$q = .50$	4.5	Position 4: 9; position 5: 10	$9 + .5(10-9) = 9.5$
$q = .75$	6.75	Position 6: 12; position 7: 15	$12 + .75(15-12) = 14.25$

Convention, not a new observation

No customer waited 5.25 minutes here. The value describes an interpolated distribution position under the default pandas rule.

Missing outcomes create a range of possible full-order rates

Stall	All orders	Recorded waits	Missing waits
A	6	4	2
B	6	6	0

$$\text{Observed late rate} = 4/10 = 40\%, \qquad \text{full late rate} \in [4/12, 6/12] = [33.3\%, 50.0\%].$$

Why a range

Among the two missing waits, zero, one or two could exceed 10 minutes. Without their outcomes or further assumptions, the full-order late rate is not determined.

- Missingness is concentrated at A, so the stalls' observed-case comparisons may be uneven.
- Determine why values are missing before choosing an imputation strategy.
- Report observed counts and missing counts with the estimate.

A bound on missing waits gives a bound on the full mean

Additional assumption

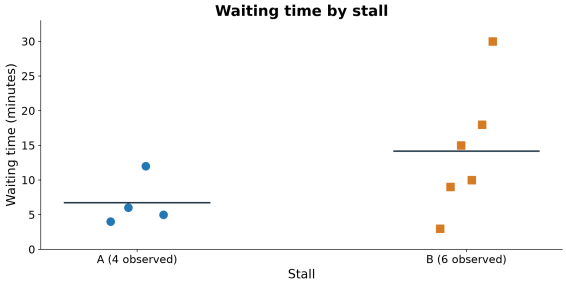
Suppose each of the two missing waits lies between 0 and 30 minutes. The ten observed waits sum to 112.

$$\frac{112+0+0}{12} \le \overline{w}_{\text{all}} \le \frac{112+30+30}{12}.$$

$$9.33 \le \overline{w}_{\text{all}} \le 14.33 \text{ min}.$$

- The observed mean 11.2 describes ten outcomes. The bounded mean describes all twelve under a stated assumption.
- If longer waits are possible, the upper bound changes.
- A transparent bound expresses uncertainty without inventing two precise measurements.

Stall comparisons need individual observations as well as averages



- A: four observed waits, mean 6.75 min.
- B: six observed waits, mean 14.17 min.
- Individual points show overlapping waits and B's long 30-minute order.
- The means summarize unequal observed counts. A also has two missing outcomes.
- A point or distribution plot reveals variation that two mean bars hide.

Scope of the statement

In this teaching sample, recorded A orders waited less on average. The records alone do not guarantee a faster future order or a population-wide stall difference.

A group comparison needs comparable operating conditions

Potential difference	What it can change	A more comparable design
Order time	Busy-period exposure	Compare similar time windows.
Dish complexity	Preparation duration	Record or group dish types.
Queue length	Work ahead of an order	Compare similar queue ranges.
Missing outcomes	Who contributes to the mean	Record reasons and report effective counts.

Association is a starting point

The lower observed A mean does not isolate a causal stall effect. A planned comparison collects conditions needed for the intended question rather than adjusts blindly for every available column.

Pooling unequal mixtures can reverse a within-period comparison

A separate 200-order illustration

These values demonstrate mixture effects. They are not additional rows in our 12-order lunch table.

Period	A count	A mean / min	B count	B mean / min
Quiet	10	2	65	3
Busy	90	10	35	11

$$\overline{w}_A = (10 \cdot 2 + 90 \cdot 10)/100 = 9.2, \qquad \overline{w}_B = (65 \cdot 3 + 35 \cdot 11)/100 = 5.8.$$

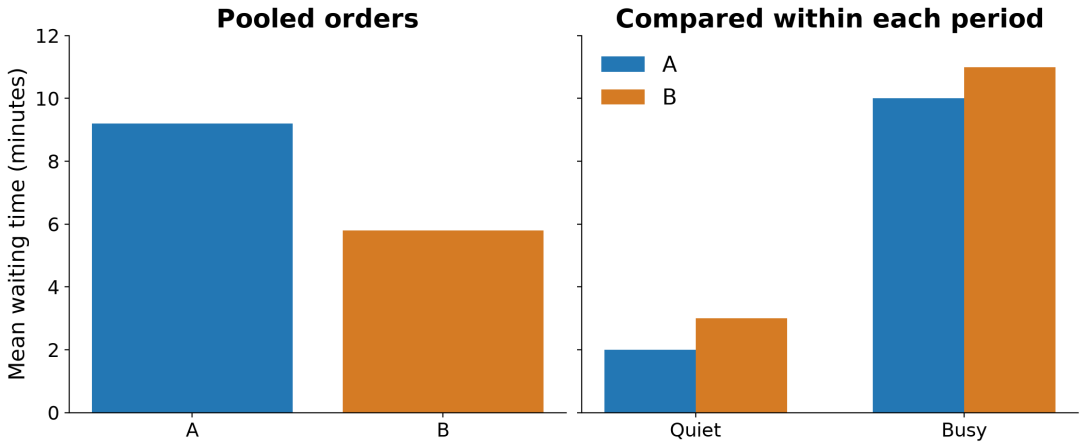
Two true statements

A is one minute faster within each period. B is faster after pooling because its orders mostly occur in the quiet period.

The lesson

Match the comparison to the question. Examine relevant groups and their proportions before interpreting a pooled mean as an effect of stall identity.

Pooling unequal mixtures can reverse a within-period comparison



A common mixture removes the composition difference

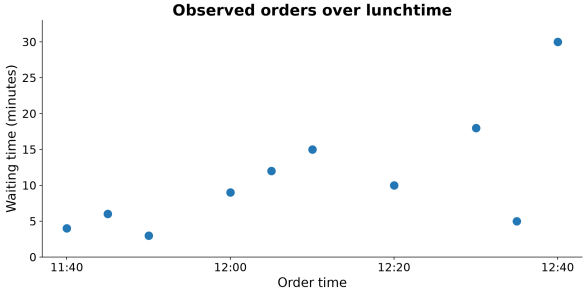
Same quiet/busy example

A has means 2 and 10; B has means 3 and 11. Compare both under a hypothetical 50% quiet and 50% busy mixture.

$$\overline{w}_A^* = .5(2) + .5(10) = 6, \qquad \overline{w}_B^* = .5(3) + .5(11) = 7.$$

- The original pooled values 9.2 and 5.8 use each stall's different observed proportions.
- The standardized values use the same specified weights, so they answer a different comparison question.
- State the target mixture and do not call the weighted result a newly observed raw average.
- Common weighting alone does not establish a causal effect.

Time order reveals a context that a pooled average removes



- The horizontal coordinate is the order timestamp.
- The vertical coordinate is realized wait in minutes.
- Each point is a different order, so a connecting line would imply an additional relationship.
- Compare patterns with ordering time, queue and stall together.
- One lunch period gives a limited time window. It cannot establish a stable daily or seasonal trend.

Question first

Use a time plot when timing might explain variation. Keep timestamps during cleaning so this question remains answerable.

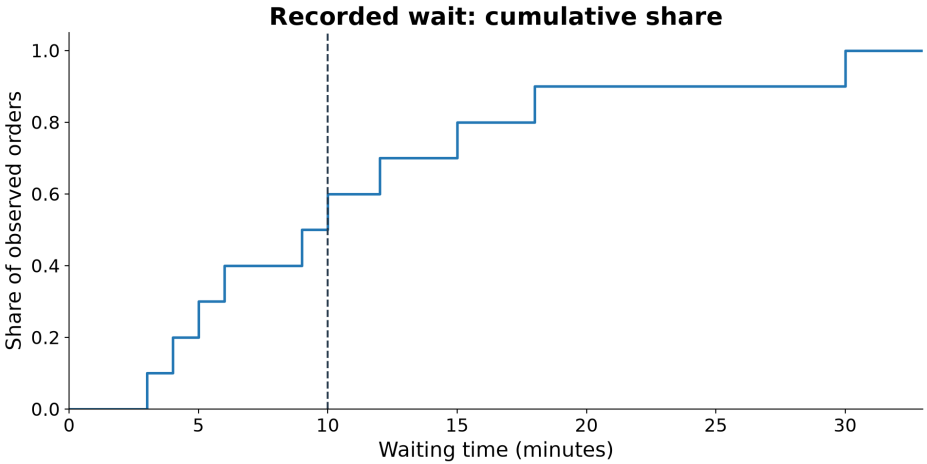
A cumulative share answers waiting-time target questions

Target t / min	Observed waits at most t	Cumulative share
6	4 of 10	0.4
10	6 of 10	0.6
15	8 of 10	0.8
30	10 of 10	1.0

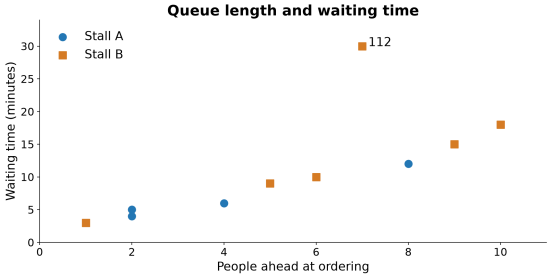
$$\widehat{F}(t) = \frac{1}{10} \sum_{i=1}^{10} \mathbb{1}\{w_i \le t\}.$$

- The share cannot decrease when the allowed time increases.
- At most 10 includes the wait of exactly 10, so the complementary share above 10 is 0.4.
- This empirical cumulative distribution describes recorded waits, not the two missing outcomes or all future lunch periods.

A cumulative share answers waiting-time target questions



A scatter plot checks how two measured variables move together



- One point is one order with an observed wait.
- Queue length is horizontal. Wait in minutes is vertical.
- The overall relationship is positive, with important exceptions.
- Order 112 waits 30 minutes despite a queue of 7.
- Color separates stalls, so one association does not erase group differences.

A compact summary

Pearson correlation for these ten complete orders is approximately $r = 0.718$. It summarizes linear association, not the effect of intervening on queue length.

A covariance calculation reveals how correlation is formed

$$q = (2, 4, 1, 5), \quad w = (4, 6, 3, 9), \quad \overline{q} = 3, \quad \overline{w} = 5.5.$$

$$\operatorname{cov}_n(q, w) = \frac{1.5 + .5 + 5 + 7}{4} = 3.5.$$

$$\operatorname{var}_n(q) = 2.5, \quad \operatorname{var}_n(w) = 5.25, \quad r = \frac{3.5}{\sqrt{2.5(5.25)}} \approx 0.966.$$

- This is the earlier four-order array exercise, not the complete ten-order coefficient 0.718.
- Covariance depends on measurement units. Normalization by both standard deviations produces unitless correlation.
- Consistent divisor choices cancel in the ratio when the same complete pairs are used.

Zero correlation can coexist with an exact nonlinear relationship

$$r = \frac{\sum_i (x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum_i (x_i - \overline{x})^2 \sum_i (y_i - \overline{y})^2}}.$$

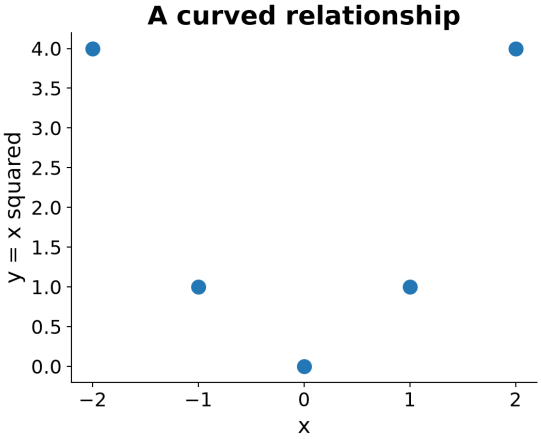
x	-2	-1	0	1	2
$y = x^2$	4	1	0	1	4

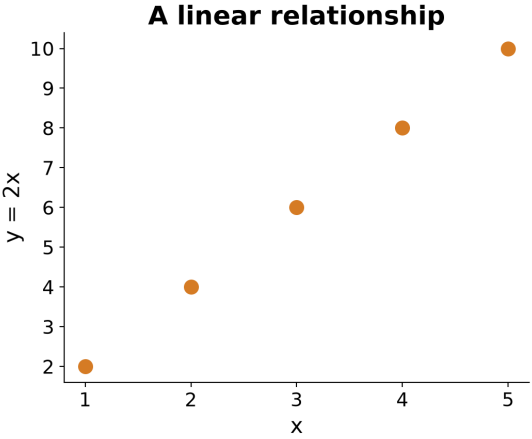
A calculation

Here $\overline{x} = 0$ and $\overline{y} = 2$. The numerator is $(-2)(2) + (-1)(-1) + 0(-2) + 1(-1) + 2(2) = 0$, so Pearson correlation is zero. Yet y is exactly determined by x.

- Pearson correlation measures linear association.
- Inspect plots for curves, outliers and subgroup mixtures.
- Association alone does not establish causality. Relevant omitted conditions can affect both variables.

Zero correlation can coexist with an exact nonlinear relationship



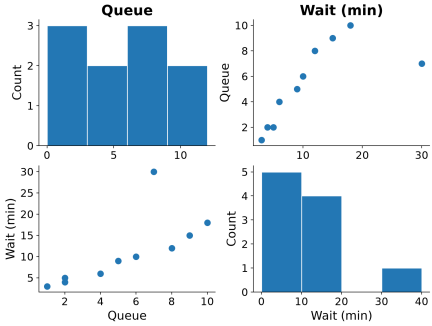


A common cause can produce a visible association

Condition	Ice-cream purchases	Swimming activity	Possible explanation
Hot day	Often higher	Often higher	Temperature affects both activities.
Cool day	Often lower	Often lower	The background condition differs.

- An association between purchases and swimming does not show that buying ice cream causes swimming.
- A third condition can influence both variables.
- For lunch data, busy-period conditions may influence queue length and preparation time together.
- Use plots to formulate explanations, then design data collection or interventions suited to distinguish them.

Pair plots connect marginal distributions and pairwise relationships



- Each diagonal panel summarizes one variable.
- Off-diagonal panels plot two variables, with the column variable on the horizontal axis and the row variable on the vertical axis.
- Lower left: queue is horizontal and wait is vertical.
- Upper right swaps those axes. It contains the same observed pairs.
- A correlation matrix is a numerical overview. Return to the relevant scatter panel to inspect shape and unusual points.

For more variables

Choose a panel by its row and column labels before interpreting the point pattern.

Exploration must respect prediction time and training-data boundaries

```
# After a deliberate train/validation/test split:
mu = train["queue"].mean()
sd = train["queue"].std(ddof=0)
z_train = (train["queue"] - mu) / sd
z_test = (test["queue"] - mu) / sd
```

- A queue measured when ordering can be an input for predicting wait. The realized wait and pickup timestamp become available later.
- Estimate preprocessing rules from training rows, then apply the same rules to new rows.
- The order table we explored supports understanding. A later predictive experiment still needs a deliberate split and an independent evaluation.
- With training mean 4 and standard deviation 2, a new queue of 8 becomes $z = 2$. It does not redefine the training mean.

A small pipeline makes information boundaries visible

```
train = orders.iloc[:6].copy()
validation = orders.iloc[6:9].copy()
test = orders.iloc[9:].copy()
center = train["queue"].mean()
train_q = train["queue"] - center
test_q = test["queue"] - center
```

- This slice-based partition illustrates six, three and three records in the cleaned 12-order table.
- The training part estimates the centering parameter; test values only use it.
- A partition does not automatically guarantee independence or representative future conditions.
- For a real waiting predictor, choose the split by the intended time, customer and operating-unit generalization.

A better recording design makes future questions answerable

Record consistently	Why it matters
Date and complete timestamps	Separate days, identify durations and retain order
Stall and queue at ordering	Describe conditions available before prediction
Order and pickup identifiers	Distinguish real orders from copied or repeated events
Missingness reason	Distinguish unrecorded wait from cancellation or immediate service
Sampling procedure	Explain whose orders and which periods the table represents

Limits of the current table

One lunch period and a small teaching sample cannot justify a statement about all students or every future day.

A concrete improvement

Record consecutive orders across several days and both busy and quiet periods, using one definition of when waiting starts and ends.

Class exercise: give a defensible answer to the lunch question

Use the cleaned teaching data

Prepare a short answer for a student deciding what the records tell them about stalls A and B.

Part of the answer	Evidence available here
Observed comparison	A mean 6.75 min from 4 waits; B mean 14.17 min from 6
Variation	Individual waits overlap; B has a 30-minute order
Missing outcomes	Two A waits are unknown
Prediction limitation	One observed mean does not guarantee the next order
What to collect next	More dates, time periods and explicit reasons for missing outcomes

Discuss the wording

What can you state directly? Which statement needs an assumption? Which would exceed the recorded evidence? Support every numerical claim with its denominator.

A conclusion exercise distinguishes three strengths of claim

Claim	Supported here?	Why
Recorded A orders averaged less wait	Yes, with counts and missingness	6.75 from 4 versus 14.17 from 6.
Every future A order will be faster	No	Means overlap and the sample covers one lunch period.
Stall A itself causes shorter waits	Not established	Operating conditions and recording processes differ.

Rewrite the answer

Keep the numerical observation. Add its sampling scope and unknown outcomes. Propose a concrete next collection step rather than replace the answer with either an absolute guarantee or a vague refusal to conclude.

Data operations support a connected explanation of the evidence

Stage	What we learned from one dataset
Numerical arrays	Shapes identify axes. Selection and sorting must keep observations intact.
Labeled tables	Names and keys preserve meaning. Types, duplicates and missing values require explicit interpretation.
Grouped summaries	Means and rates depend on the selected rows and denominator.
Exploration	Plots reveal distribution, association and exceptions that one summary can hide.
Conclusions	State the observed result, its missing information and the population represented.

Bridge to L03

The next lecture models class distributions and combines evidence with prior probabilities. Reliable calculation begins with the same discipline: know what each row, variable and probability refers to.